EECS 4412 Project (GitHub link as a supporting resource: link)

Team Description

Teammates	*Student Name*	*Student ID*	*eClass Email*
1	Shayan Rabiee	217761263	shayan26@my.yorku.ca
2	Dylan Smith	217285287	dsmitty@my.yorku.ca
3	Hanifah Lameed	219714815	hani5115@my.yorku.ca

Team Statements

Shayan Rabiee

I, **Shayan Rabiee**, Student ID **217761263**, acknowledge that I contributed at least 30% of my time and effort to preparing this report and the work discussed herein.

Signature:



Dylan Smith

I, **Dylan Smith**, Student ID **217285287**, acknowledge that I have contributed at least 30% of my time and effort to the preparation of this report and the work discussed herein.

Signature:



Hanifah Lameed

I, **Hanifah Lameed**, Student ID **219714815**, acknowledge that I contributed at least 30% of my time and effort to preparing this report and the work discussed herein.

Signature: HanifahLameed

Table of Contents

EECS 4412 Project (GitHub link as a supporting resource: link) **1**
Team Description **1**
Team Statements **1**
Shayan Rabiee 1
Dylan Smith 1
Task 1: Principal Component Analysis **3**
1.1 Data Preprocessing 3
1.2 PCA Loadings 4
1.3 Explained Variance 5
1.4 Visualization of the Principal Components 7
Task 2: Classification **8**
2.1 Training and Test Data 9
2.2 Decision Tree Classifier 9
2.3 Naïve Bayes Classifier 16
2.4 Classifier Comparison 20
Task 3: Clustering Analysis **21**
3.1 Preprocessing and Method 21
3.2 SSE Results 21
3.3 Comparison with Classification 22
Task 4: Progress on Our Objectives **23**
4.1 Progress on the Original Research Questions 23
4.2 Changes to Our Research Questions 24
4.3 Revised Research Question 5 25

Task 1: Principal Component Analysis

For the Principal Component Analysis (PCA), I used data from the Trip and Household tables of the 2017 National Household Travel Survey (NHTS). The assignment permits using more than one table/object type for this task. EECS4412--F26--P02

The Trip table was joined with selected Household-level attributes using the common HOUSEID field. After the merge, the following six numerical dimensions were selected for the PCA:

Dimension	Description
TRPMILES	**Trip distance in miles**
TRVLCMIN	**Trip travel duration in minutes**
R_AGE	**Age of the traveller**
HHSIZE	**Number of people in the household**
HHVEHCNT	**Number of vehicles available to the household**
WRKCOUNT	**Number of workers in the household**

1.1 Data Preprocessing

Before performing PCA, we converted all six selected dimensions to numeric values. We removed missing observations and excluded negative values representing NHTS special-response or missing-data codes. After preprocessing, 920,406 valid observations remained for PCA.

Because the selected attributes have different numerical scales, we standardized the data using Standard Scaler. This transformed each dimension so differences in units, such as miles, minutes, age, and household counts, would not let one attribute dominate PCA simply because of its magnitude.

We then performed PCA on the standardized data and retained the three principal components that explained the most variance.

1.2 PCA Loadings

The following table shows each original dimension's contribution to the three selected principal components.

Table 1: PCA Component Loadings

Original Dimension	PC1	PC2	PC3
TRPMILES	0.023457	0.705649	-0.040359
TRVLCMIN	0.027595	0.705585	-0.033618
R_AGE	-0.494062	0.041141	0.565694
HHSIZE	0.555034	-0.036864	-0.117159
HHVEHCNT	0.380420	0.031421	0.811676
WRKCOUNT	0.549372	-0.013087	0.068463

The loadings show that PC1 is mainly influenced by household size (HHSIZE), number of workers (WRKCOUNT), traveller age (R_AGE), and household vehicle count (HHVEHCNT). HHSIZE and WRKCOUNT have strong positive loadings, while R_AGE has a negative loading. Therefore, PC1 can generally be interpreted as representing household composition and workforce characteristics.

PC2 is almost completely dominated by TRPMILES and TRVLCMIN, both with loadings of approximately 0.706. This indicates that PC2 primarily represents trip magnitude, with higher PC2 scores corresponding to longer trips in distance and travel time.

PC3 is primarily influenced by HHVEHCNT, with a loading of approximately 0.812, followed by R_AGE, with a loading of approximately 0.566. Therefore, PC3 primarily captures differences in household vehicle availability and traveller age.

1.3 Explained Variance

The three selected components explain the following proportions of the total variance:

Principal Component	Explained Variance
PC1	36.58%
PC2	24.85%
PC3	15.23%
Total	76.66%

Together, PC1, PC2, and PC3 explain 76.66% of the total variance contained in the six original dimensions. Therefore, the PCA reduces the dataset from six dimensions to three components while retaining more than three-quarters of the original data's variance.

```
Trip dataset loaded successfully.
Shape: (923572, 4)

Clean PCA dataset:
   TRPMILES  TRVLCMIN  R_AGE  HHSIZE  HHVEHCNT  WRKCOUNT
0     5.244      15.0   67.0       3         5         1
1     5.149      20.0   67.0       3         5         1
2    84.004     120.0   66.0       3         5         1
3    81.628     150.0   66.0       3         5         1
4     2.250      15.0   28.0       3         5         1

Number of valid observations: 920406

====================================
PCA LOADINGS
====================================
                PC1       PC2       PC3
TRPMILES   0.023457  0.705649 -0.040359
TRVLCMIN   0.027595  0.705585 -0.033618
R_AGE     -0.494062  0.041141  0.565694
HHSIZE     0.555034 -0.036864 -0.117159
HHVEHCNT   0.380420  0.031421  0.811676
WRKCOUNT   0.549372 -0.013087  0.068463
```

```
========================================
PCA EXPLAINED VARIANCE
========================================
PC1: 36.58 %
PC2: 24.85 %
PC3: 15.23 %
Total: 76.66 %

First PCA observations:
        PC1       PC2       PC3
0  0.450096 -0.100165  2.327153
1  0.454278  0.006611  2.322075
2  0.590245  2.990987  2.144323
3  0.614704  3.612613  2.114944
4  1.386799 -0.209798  1.255245
```

Figure 1: Explained Variance of Principal Components

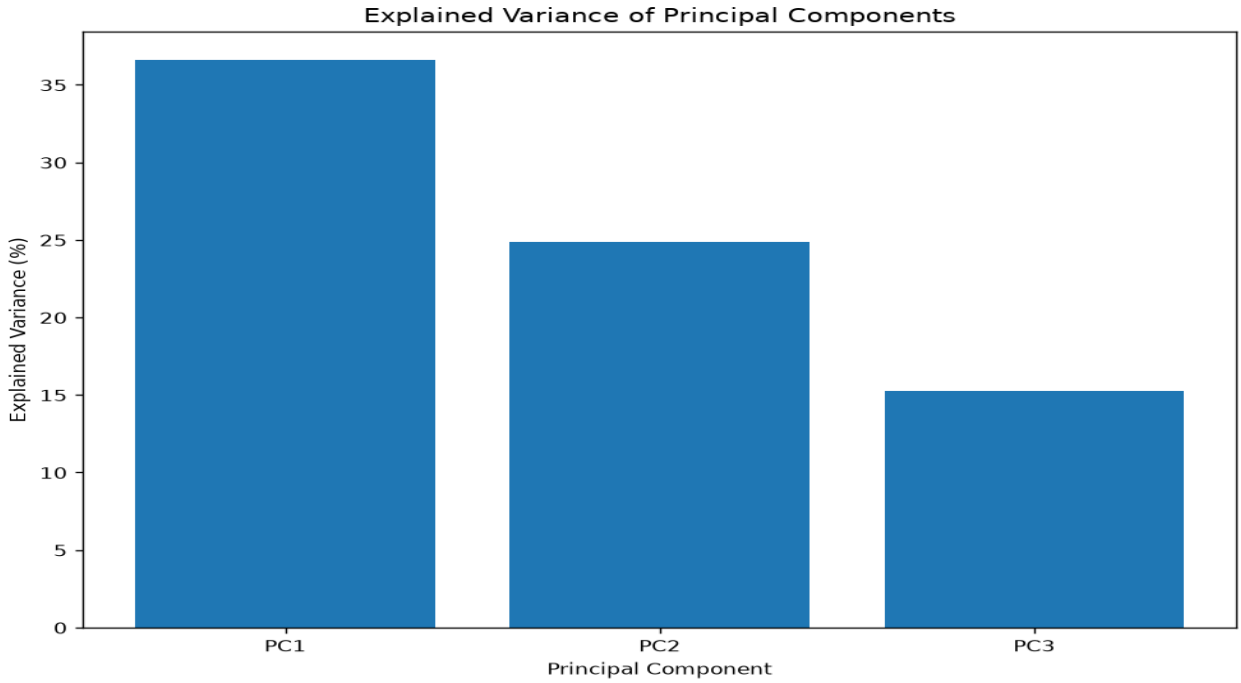


Figure 1. Explained variance of the three selected principal components.

Figure 1 shows that PC1 accounts for the greatest proportion of variance (36.58%), PC2 the next largest (24.85%) and PC3 the smallest (15.23%). The decreasing contributions indicate that PC1 captures the most information in the original set of standardized variables.

1.4 Visualization of the Principal Components

A scatter plot of PC1 against PC2 shows the observations projected into the reduced PCA space.

Figure 2: PCA Projection — PC1 vs. PC2

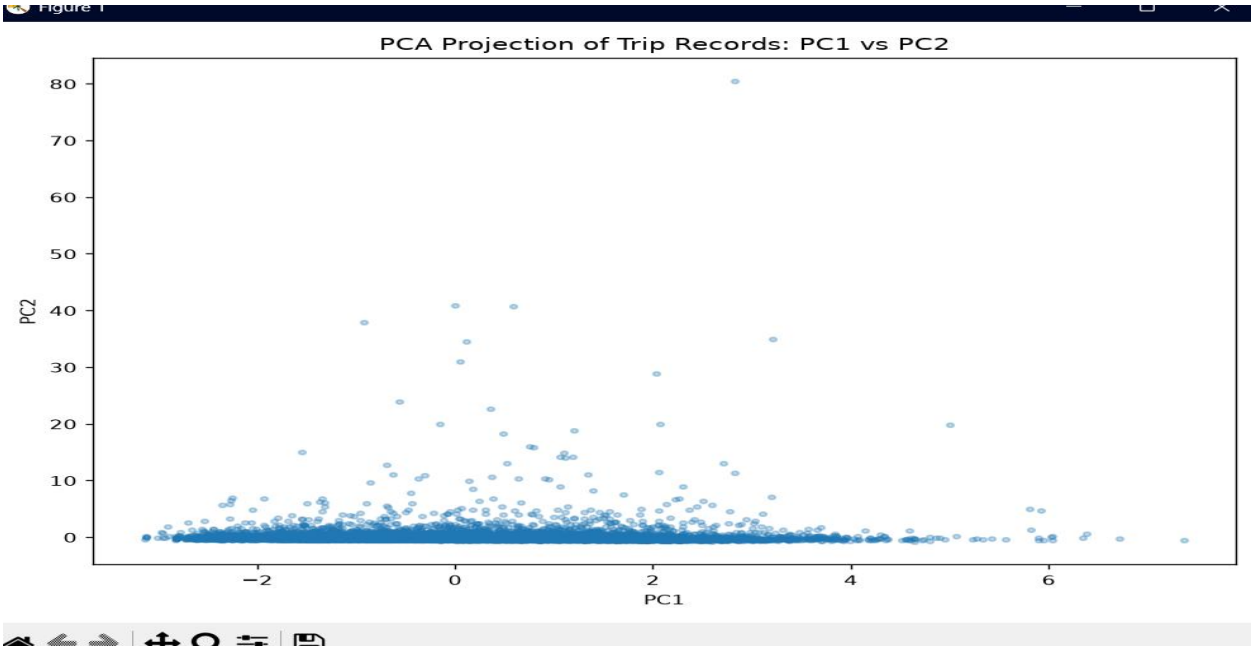


Figure 2. Projection of trip and household observations onto the first and second principal components.

Figure 2 shows that most observations cluster in a relatively compact region, while a smaller number extend to much higher PC2 values. Since PC2 is strongly influenced by TRPMILES and TRVLCMIN, these observations likely correspond to trips with unusually large distances or travel durations.

For readability, the scatter plot used a random sample of up to 10,000 observations. This sampling applied only to the visualization; PCA was calculated on the full cleaned dataset of 920,406 observations.

Task 2: Classification

Within assignment one, we proposed the following two classification questions:

- **1. How accurately can household circumstances, traveller characteristics, trip purpose, and day of the week predict a traveler's mode of transportation?**
- **4. How accurately can household characteristics classify households by the number of vehicles they own?**

For task 2, we chose question 4: How accurately can household characteristics classify households by the number of vehicles they own?

To do so, we need to look at the household object. The HHVEHCNT column shows the number of vehicles per household. Before removing records with missing predictor values, the classes had the following counts:

1. **HHVEHCNT = 0** : 0 Vehicles: 6,249 households
2. **HHVEHCNT = 1** : 1 vehicle: 41,534
3. **HHVEHCNT = 2** : 2 vehicles: 49,936
4. **HHVEHCNT = 3+** : 3+ vehicles: 31,977

The household characteristics in question are:

- **HHSIZE**: Household size
- **DRVRCNT**: Driver count of the household
- **WRKCOUNT**: number of workers per household, may have connections to the number of vehicles needed
- **HHFAMINC**: household income category, may reflect how many vehicles they own
- **HOMEOWN**: homeownership
- **YOUNGCHILD**: Young children in the house
- **URBAN**: classification of whether the house is in an urban area or not

So, putting these together, we can then come up with the more detailed question:

- **Given a household's size, number of drivers, number of workers, income group, homeownership status, number of young children and urban classification, can we predict whether the household has 0, 1, 2 or 3+ vehicles?**

2.1 Training and Test Data

- We used k-fold cross-validation with a holdout test set. Of the 125,298 households, we used 100,238 for training and 25,060 for testing.
- We set aside a stratified random 20% of the records as our test set (random_state = 42) and used k = 5 for cross-validation. We split the remaining 80% into five folds, using four for training and one for validation.
- We repeated this five times, with each fold acting as the validation set once.
- We averaged the five validation accuracies for each setting.
- Once this was complete, we trained the decision tree and Categorical Naïve Bayes classifiers on the full training set using the settings we selected. We then evaluated both on the same held-out test set.

2.2 Decision Tree Classifier

We went through all the fields using the derived variables. We checked the values and missing-response codes for all seven predictors.

Attribute	Description	Measurement Type	Values
HHSIZE	Number of household members	Ratio: numerical count	Observed range: 1–13
DRVRCNT	Number of drivers in the household	Ratio: numerical count	Observed range: 0–9
WRKCOUNT	Number of workers in the household	Ratio: numerical count	Observed range: 0–7
YOUNGCHILD	Number of household members aged 0–4	Ratio: numerical count	Observed range: 0–5
HHFAMINC	Annual household income category	Ordinal	Codes 1–11, ordered from lowest to highest income
HOMEOWN	Household homeownership arrangement	Nominal	1 = Own; 2 = Rent; 97 = Some other arrangement
URBAN	Urban-area classification of the household's address	Nominal	1 = Urban area; 2 = Urban cluster; 3 = Area surrounded by urban areas; 4 = Not in an urban area

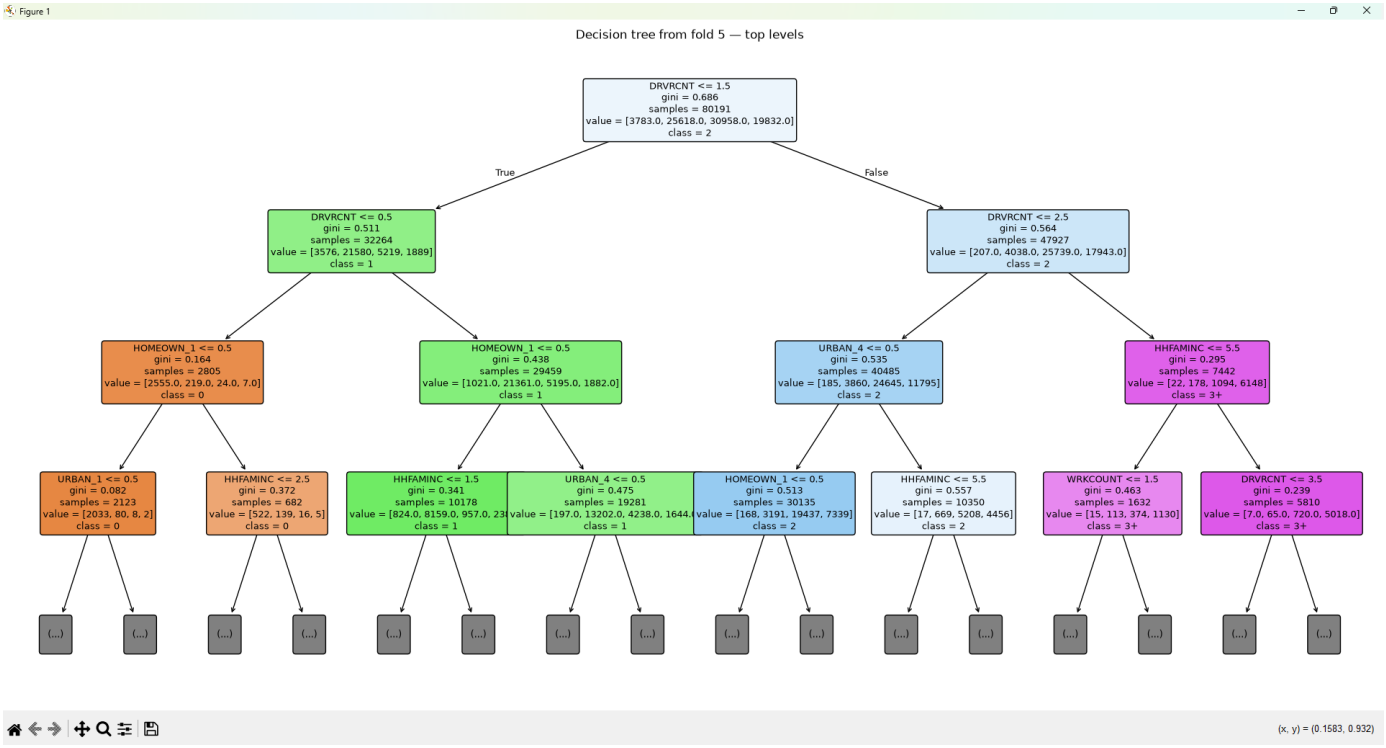
Note: The observed ranges described the values in this dataset. We handled negative missing-response codes in HHFAMINC and HOMEOWN separately.

Following the approach in Module 3, we removed records with missing predictor values. Across these fields, 4,398 households had missing data, as shown in our GitHub report. This left us with 125,298 households.

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2> py .\task2_split.py
Households removed: 4398
Households remaining: 125298
```

```
households in each class:
VEHICLE_CLASS
0      5911
1     40027
2     48372
3+    30988
Name: count, dtype: int64
```

After we cleaned the data, we trained the tree. At each node, the tree selected the split that produced the largest reduction in Gini impurity, weighting the child nodes by their size.




```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2> py .\task2_createfolds.py
Fold 1 validation accuracy: 67.90%
Fold 1: tree trained
Fold 1: 80190 training households, 20048 validation households
tree depth: 21
Fold 2 validation accuracy: 67.74%
Fold 2: tree trained
Fold 2: 80190 training households, 20048 validation households
tree depth: 21
Fold 3 validation accuracy: 67.51%
Fold 3: tree trained
Fold 3: 80190 training households, 20048 validation households
tree depth: 21
Fold 4 validation accuracy: 67.60%
Fold 4: tree trained
Fold 4: 80191 training households, 20047 validation households
tree depth: 21
Fold 5 validation accuracy: 68.42%
Fold 5: tree trained
Fold 5: 80191 training households, 20047 validation households
tree depth: 22
Mean 5-fold validation accuracy: 67.83%
```

So we started at 67.83% mean accuracy with the model using unrestricted depth. With unrestricted depth, the gap between training and validation accuracy was 2.55 percentage points.

Using a max_depth of 10:

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2> py .\task2_createfolds.py
Fold 1 validation accuracy: 68.59%
Fold 1: training=69.41%, validation=68.59%
Fold 1: tree trained
Fold 1: 80190 training households, 20048 validation households
tree depth: 10
Fold 2 validation accuracy: 68.66%
Fold 2: training=69.39%, validation=68.66%
Fold 2: tree trained
Fold 2: 80190 training households, 20048 validation households
tree depth: 10
Fold 3 validation accuracy: 68.07%
Fold 3: training=69.51%, validation=68.07%
Fold 3: tree trained
Fold 3: 80190 training households, 20048 validation households
tree depth: 10
Fold 4 validation accuracy: 68.29%
Fold 4: training=69.49%, validation=68.29%
Fold 4: tree trained
Fold 4: 80191 training households, 20047 validation households
tree depth: 10
Fold 5 validation accuracy: 69.07%
Fold 5: training=69.29%, validation=69.07%
Fold 5: tree trained
Fold 5: 80191 training households, 20047 validation households
tree depth: 10
Mean 5-fold validation accuracy: 68.53%
```

Limiting the depth reduced the gap to 0.89 percentage points and improved mean validation accuracy by 0.7 percentage points. The smaller gap suggested reduced overfitting.

Using a max_depth of 5:

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2> py .\task2_createfolds.py
Fold 1 validation accuracy: 68.76%
Fold 1: training=68.42%, validation=68.76%
Fold 1: tree trained
Fold 1: 80190 training households, 20048 validation households
tree depth: 5
Fold 2 validation accuracy: 68.71%
Fold 2: training=68.44%, validation=68.71%
Fold 2: tree trained
Fold 2: 80190 training households, 20048 validation households
tree depth: 5
Fold 3 validation accuracy: 68.01%
Fold 3: training=68.61%, validation=68.01%
Fold 3: tree trained
Fold 3: 80190 training households, 20048 validation households
tree depth: 5
Fold 4 validation accuracy: 68.12%
Fold 4: training=68.68%, validation=68.12%
Fold 4: tree trained
Fold 4: 80191 training households, 20047 validation households
tree depth: 5
Fold 5 validation accuracy: 68.75%
Fold 5: training=68.43%, validation=68.75%
Fold 5: tree trained
Fold 5: 80191 training households, 20047 validation households
tree depth: 5
Mean 5-fold validation accuracy: 68.47%
```

Mean validation accuracy was 0.06 percentage points higher at depth 10 than at depth 5. Depth 15 performed worse (67.96% validation accuracy, 70.22% training accuracy).

- **Depth 5:** 68.47%
- **Depth 10:** 68.53%
- **Depth 15:** 67.96%
- **Unrestricted:** 67.83%

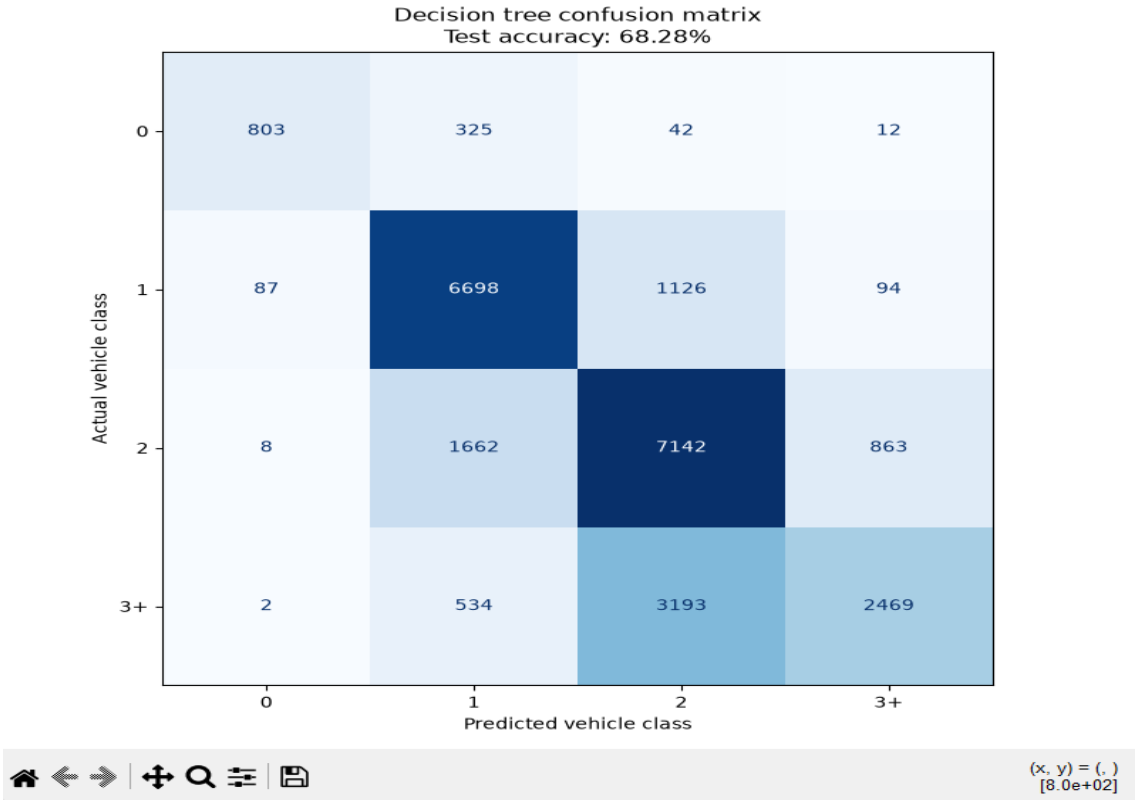
We settled on a max_depth of 10 because it gave the highest mean validation accuracy. Its training–validation gap was 0.89 percentage points, compared with 2.55 percentage points for the unrestricted tree, suggesting reduced overfitting.

We trained the depth-10 tree on all 100,238 training households and saved it as task2_final_tree.pkl. We then evaluated it on the 25,060 households in the held-out test set.

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2> py .\task2_full_training_set_test.py
Test households: 25060
Tree depth: 10
Final test accuracy: 68.28%
```

Our final decision tree test accuracy was 68.28% (just 0.25 percentage points below the decision tree validation average).

To visualize the model's performance, we used a confusion matrix to compare the model's predicted classifications with the actual labels (test set).

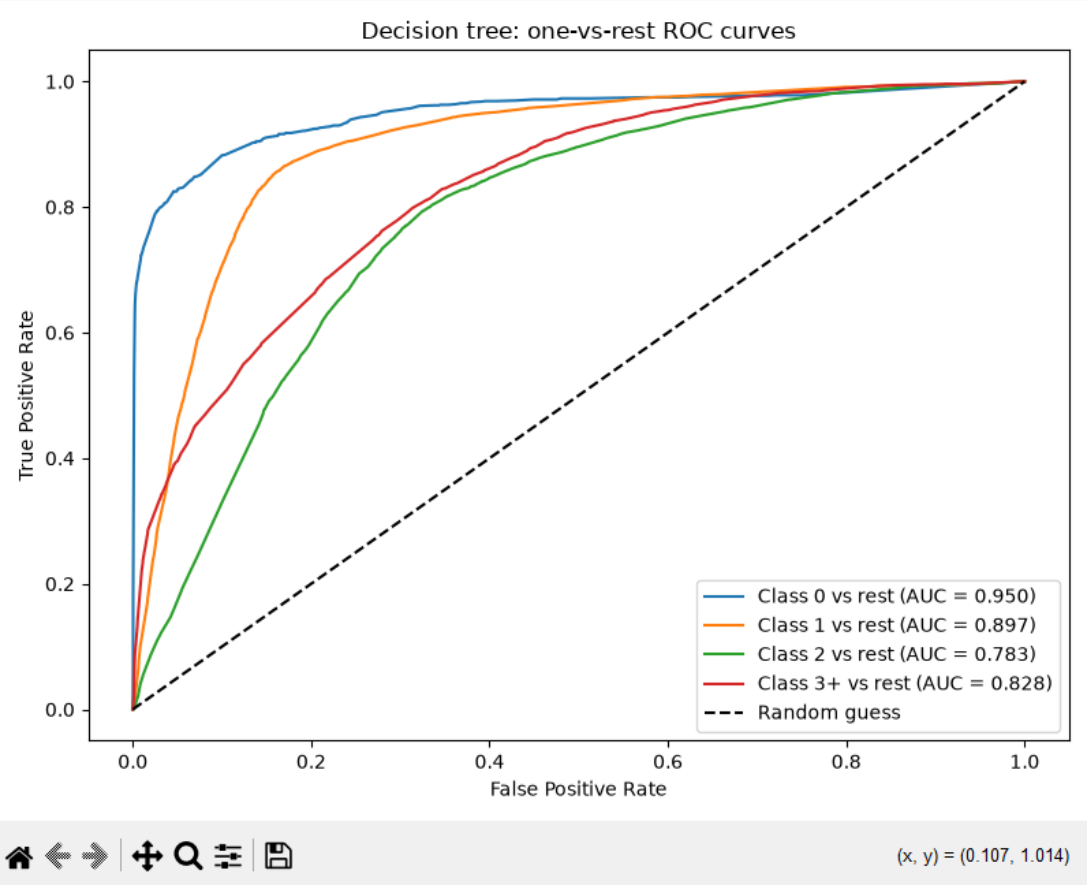


We used the confusion matrix to see which vehicle classes the tree confused most often. While we correctly predicted 2,469 households as 3-plus, we incorrectly predicted 3,193 as 2. Thus, of 6,198 actual 3-plus households, we correctly identified only 39.84%.

The confusion matrix suggested that the tree had trouble distinguishing households with three or more vehicles from those with two vehicles using our seven predictors.

Figure 3. Decision tree one-vs-rest ROC curves.

```
Class probabilities for the first five households:
          0         1         2         3+
0  0.012346  0.604938  0.234568  0.148148
1  0.001226  0.031873  0.649402  0.317499
2  0.003638  0.044179  0.566528  0.385655
3  0.001676  0.049907  0.666108  0.282309
4  0.003731  0.782183  0.159981  0.054104
```



All four one-vs-rest ROC curves had an AUC above 0.5, the value for random guessing.

2.3 Naïve Bayes Classifier

Our Naïve Bayes classifier compared the probabilities of the four vehicle ownership classes using the seven household predictors. It assumed the predictors were conditionally independent within each class.

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)}$$

We selected Categorical Naïve Bayes because our predictors took finite, discrete values, as shown in the predictor table above.

Scikit-learn's Categorical Naive Bayes was used: CategoricalNB — scikit-learn 1.9.1 documentation

We used the same training and test sets and the same five-fold cross-validation procedure we used for the decision tree.

When creating the CategoricalNB, we ensured the values started at 0 and supplied the correct min_categories.

Using an alpha value of 1, we got the following:

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2\Naive_bayes> py .\task2_p2_1_createfolds.py
Fold 1 Naive Bayes classifier parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 1 validation accuracy: 67.31%
Fold 1: training=67.43%, validation=67.31%
Fold 1: Naive Bayes classifier trained
Fold 1: 80190 training households, 20048 validation households
Fold 2 Naive Bayes classifier parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 2 validation accuracy: 67.40%
Fold 2: training=67.44%, validation=67.40%
Fold 2: Naive Bayes classifier trained
Fold 2: 80190 training households, 20048 validation households
Fold 3 Naive Bayes classifier parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 3 validation accuracy: 67.32%
Fold 3: training=67.46%, validation=67.32%
Fold 3: Naive Bayes classifier trained
Fold 3: 80190 training households, 20048 validation households
Fold 4 Naive Bayes classifier parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 4 validation accuracy: 67.15%
Fold 4: training=67.48%, validation=67.15%
Fold 4: Naive Bayes classifier trained
Fold 4: 80191 training households, 20047 validation households
Fold 5 Naive Bayes classifier parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 5 validation accuracy: 67.83%
Fold 5: training=67.41%, validation=67.83%
Fold 5: Naive Bayes classifier trained
Fold 5: 80191 training households, 20047 validation households
Mean 5-fold validation accuracy: 67.40%
```

- Mean training accuracy: 67.44%
- Mean validation accuracy: 67.40%
- Gap: 0.04 percentage points

Using an alpha of 0.1, we got:

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2\Naive_bayes> py .\task2_p2_1_createfolds.py
Fold 1 Naive Bayes classifier parameters:
{'alpha': 0.1, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 1 validation accuracy: 67.32%
Fold 1: training=67.43%, validation=67.32%
Fold 1: Naive Bayes classifier trained
Fold 1: 80190 training households, 20048 validation households
Fold 2 Naive Bayes classifier parameters:
{'alpha': 0.1, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 2 validation accuracy: 67.41%
Fold 2: training=67.44%, validation=67.41%
Fold 2: Naive Bayes classifier trained
Fold 2: 80190 training households, 20048 validation households
Fold 3 Naive Bayes classifier parameters:
{'alpha': 0.1, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 3 validation accuracy: 67.31%
Fold 3: training=67.47%, validation=67.31%
Fold 3: Naive Bayes classifier trained
Fold 3: 80190 training households, 20048 validation households
Fold 4 Naive Bayes classifier parameters:
{'alpha': 0.1, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 4 validation accuracy: 67.17%
Fold 4: training=67.45%, validation=67.17%
Fold 4: Naive Bayes classifier trained
Fold 4: 80191 training households, 20047 validation households
Fold 5 Naive Bayes classifier parameters:
{'alpha': 0.1, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 5 validation accuracy: 67.84%
Fold 5: training=67.42%, validation=67.84%
Fold 5: Naive Bayes classifier trained
Fold 5: 80191 training households, 20047 validation households
Mean 5-fold validation accuracy: 67.41%
```

- Mean training accuracy: 67.44%
- Mean validation accuracy: 67.41%
- Gap: 0.03 percentage points

Alpha = 10

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2\Naive_bayes> py .\task2_p2_1_createfolds.py
Fold 1 Naive Bayes classifier parameters:
{'alpha': 10, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 1 validation accuracy: 67.32%
Fold 1: training=67.41%, validation=67.32%
Fold 1: Naive Bayes classifier trained
Fold 1: 80190 training households, 20048 validation households
Fold 2 Naive Bayes classifier parameters:
{'alpha': 10, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 2 validation accuracy: 67.33%
Fold 2: training=67.38%, validation=67.33%
Fold 2: Naive Bayes classifier trained
Fold 2: 80190 training households, 20048 validation households
Fold 3 Naive Bayes classifier parameters:
{'alpha': 10, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 3 validation accuracy: 67.30%
Fold 3: training=67.47%, validation=67.30%
Fold 3: Naive Bayes classifier trained
Fold 3: 80190 training households, 20048 validation households
Fold 4 Naive Bayes classifier parameters:
{'alpha': 10, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 4 validation accuracy: 67.13%
Fold 4: training=67.42%, validation=67.13%
Fold 4: Naive Bayes classifier trained
Fold 4: 80191 training households, 20047 validation households
Fold 5 Naive Bayes classifier parameters:
{'alpha': 10, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 5 validation accuracy: 67.79%
Fold 5: training=67.34%, validation=67.79%
Fold 5: Naive Bayes classifier trained
Fold 5: 80191 training households, 20047 validation households
Mean 5-fold validation accuracy: 67.37%
```

- Mean training accuracy: 67.40%
- Mean validation accuracy: 67.37%
- Gap: 0.03 percentage points
- Lower training accuracy than a = 1.

Finally, we tested alpha = 0.001:

```
Fold 1 Naive Bayes classifier parameters:
{'alpha': 0.001, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 1 validation accuracy: 67.32%
Fold 1: training=67.43%, validation=67.32%
Fold 1: Naive Bayes classifier trained
Fold 1: 80190 training households, 20048 validation households
Fold 2 Naive Bayes classifier parameters:
{'alpha': 0.001, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 2 validation accuracy: 67.41%
Fold 2: training=67.45%, validation=67.41%
Fold 2: Naive Bayes classifier trained
Fold 2: 80190 training households, 20048 validation households
Fold 3 Naive Bayes classifier parameters:
{'alpha': 0.001, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 3 validation accuracy: 67.31%
Fold 3: training=67.47%, validation=67.31%
Fold 3: Naive Bayes classifier trained
Fold 3: 80190 training households, 20048 validation households
Fold 4 Naive Bayes classifier parameters:
{'alpha': 0.001, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 4 validation accuracy: 67.17%
Fold 4: training=67.46%, validation=67.17%
Fold 4: Naive Bayes classifier trained
Fold 4: 80191 training households, 20047 validation households
Fold 5 Naive Bayes classifier parameters:
{'alpha': 0.001, 'class_prior': None, 'fit_prior': True, 'force_alpha': True, 'min_categories': [13, 10, 8, 6, 11, 3, 4]}
Fold 5 validation accuracy: 67.84%
Fold 5: training=67.42%, validation=67.84%
Fold 5: Naive Bayes classifier trained
Fold 5: 80191 training households, 20047 validation households
Mean 5-fold validation accuracy: 67.41%
```

- Mean training accuracy: 67.45%
- Mean validation accuracy: 67.41%
- Gap: 0.04 percentage points
- Alpha values of 0.1 and 0.001 tied for the highest mean validation accuracy at 67.41%. We chose 0.1.

Using an alpha of 0.1 and aligning the same values to start at zero, we trained the categorical Naïve Bayes on all 100,238 training households.

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2\Naive_bayes> py .\task2_p2_2_train_final.py
Training households: 100238
Final model saved to: C:\Users\dnsh\EECS-4412-Project\ass2part2\task2_final_naive_bayes.pkl
```

The final model achieved 67.28% accuracy on the held-out test set.

```
PS C:\Users\dnsh\EECS-4412-Project\ass2part2\Naive_bayes> py .\task2_p2_3_test_final.py
Test households: 25060
Alpha: 0.1
Final Naive Bayes test accuracy: 67.28%
```

Figure 4. Categorical Naïve Bayes confusion matrix on the held-out test set.

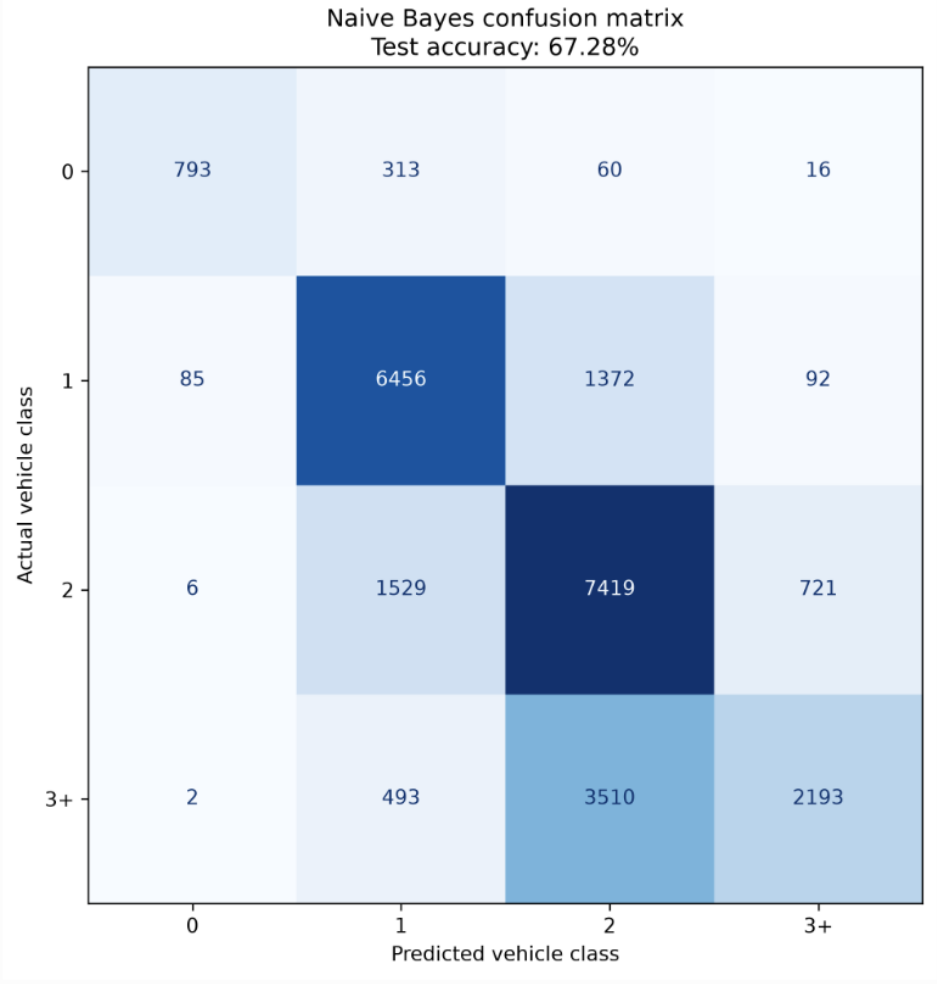
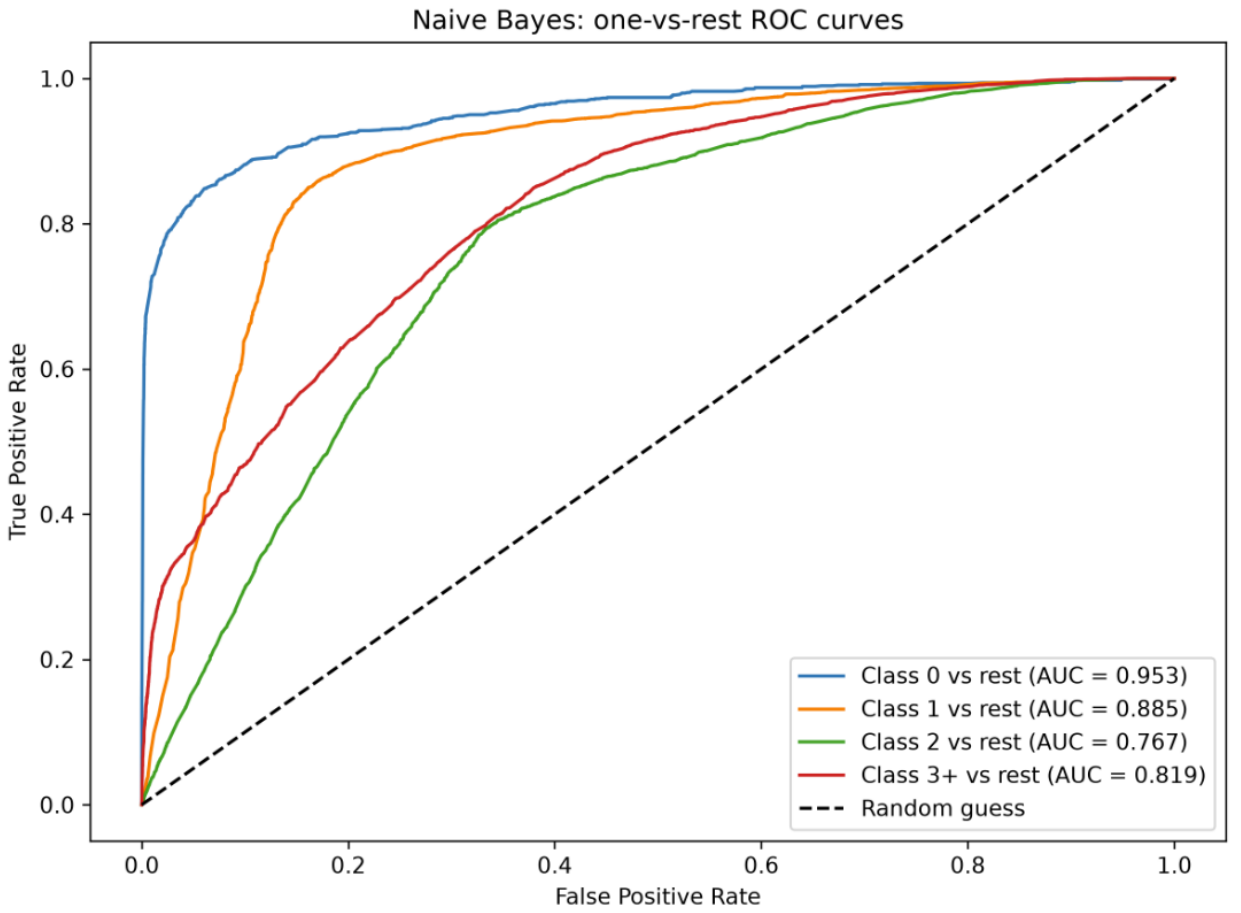


Figure 5. Categorical Naïve Bayes one-vs-rest ROC curves.



2.4 Classifier Comparison

- Decision tree: 68.28% accuracy
- Categorical Naïve Bayes: 67.28% accuracy

The decision tree's test accuracy was 1 percentage point higher.

It also had a higher AUC for classes 1, 2, and 3+, while Categorical Naïve Bayes had a slightly higher AUC for class 0.

Task 3: Clustering Analysis

3.1 Preprocessing and Method

We used the same household object type and the same seven predictors as in the previous classification task (HHSIZE, DRVRCNT, WRKCOUNT, YOUNGCHILD, HHFAMINC, HOMEOWN and URBAN). We combined the cleaned Task 2 training and test records for the descriptive clustering analysis. We standardized the numeric predictors and one-hot encoded HOMEOWN and URBAN. We treated HHFAMINC as an ordinal variable (with equal spacing between category codes). We excluded HOUSEID, as it is an identifier. We excluded HHVEHCNT and VEHICLE_CLASS because they are effectively the classification target. We ran K-means with k=3, 4, and 5, using 20 initializations and a random seed of 42. The k=4 run corresponds to the four vehicle ownership classes used in Task 2.

3.2 SSE Results

Number of clusters	SSE
3	413,388.26
4	360,377.71
5	323,705.25

```
SSE RESULTS
 k           SSE
 3 413388.258579
 4 360377.713424
 5 323705.252499

Lowest observed SSE: k = 5

HELD-OUT TEST COMPARISON
                                method  accuracy  macro_F1      ARI      NMI
K-means k=3, training-majority mapping  0.515004  0.301483 0.149638 0.165040
K-means k=4, training-majority mapping  0.556185  0.425994 0.174416 0.193902
K-means k=5, training-majority mapping  0.614605  0.444657 0.185110 0.198420
                  Decision tree baseline  0.682841  0.684013      NaN      NaN
          Gaussian Naive Bayes baseline  0.437111  0.402110      NaN      NaN
```

Among the tested values, **(k=5) produced the lowest SSE**, indicating the smallest total squared distance between observations and their assigned centroids. SSE decreased by approximately 12.82% from (k=3) to (k=4), and by another 10.18% from (k=4) to (k=5). Increasing the number of clusters normally reduces SSE because more centroids can represent the observations. Therefore, these results show that (k=5) yields the lowest observed SSE among the tested options, but they do not show that five clusters provide the most meaningful household grouping.

3.3 Comparison with Classification

We fitted both the preprocessing and the models separately using only the training observations and evaluated them on the held-out test set. We assigned each k-means cluster the most frequent training-set vehicle class, then applied this mapping to the test-set predictions.

Method	Test accuracy	Macro-F1
K-means (k=3)	51.50%	0.3015
K-means (k=4)	55.62%	0.4260
K-means (k=5)	61.46%	0.4447
Decision tree baseline	68.28%	0.6840
Categorical Naïve Bayes (Task 2)	67.28%	0.6712
Gaussian Naïve Bayes baseline	43.71%	0.4021

The decision tree achieved the highest accuracy and macro-F1. Its accuracy exceeded the strongest clustering approach (k=5) by approximately 6.82 percentage points. Although (k=5) outperformed Gaussian Naïve Bayes on both measures, its lower macro-F1 relative to the decision tree indicates weaker performance when the four classes receive equal weight. Both Task 2 classifiers outperformed k-means in predicting vehicle ownership. We also included Gaussian Naïve Bayes as an additional baseline.

For (k=5), the adjusted Rand index was 0.1851 and the normalized mutual information was 0.1984. That's pretty poor agreement between the clusters and the vehicle ownership classes. These values indicate limited agreement between the clusters and vehicle-ownership classes. K-means groups households by similarity in the selected predictors rather than directly optimizing vehicle-class prediction.

The decision tree was the better approach for predicting the vehicle ownership classes in this instance. K-means is still useful for identifying household clusters. The clusters showed limited agreement with vehicle ownership classes in this analysis

Task 4: Progress on Our Objectives

4.1 Progress on the Original Research Questions

Phase 1 asked five questions about household characteristics, vehicle ownership, and travel behaviour. Phase 2 aimed to predict household vehicle ownership, cluster households based on their characteristics, and apply PCA to the trip and household data to identify patterns.

1. Predicting mode of transport: In Phase 1, we looked at the distribution of the various modes of transport in the dataset; in Phase 2, we gained experience in data preparation and in the evaluation of classifiers through the analysis of vehicle ownership; we can then go on to apply this process to mode of transport prediction.

2. Predicting trip duration: PCA showed that both trip distance and trip duration loaded strongly on the same component, which provides a good basis for looking at trip duration. The next stage is to see whether adding mode of travel, trip purpose, and day of the week improves prediction over distance alone.

3. An explanation of annual vehicle mileage: In the first phase, we looked at the patterns around annual mileage (e.g. the relationship between vehicle age and mileage). We can now look at the relationship between vehicle type, number of workers, household income, and mileage.

4. The classification of ownership of vehicles in households: To look at this, we used seven household characteristics to predict whether a household had 0, 1, 2, or 3+ vehicles.

5. The identification of household travel-day profiles: We made some progress by splitting the households into groups according to their demographic and socio-economic characteristics, as the analysis was at the household level, so we've amended the question to reflect the variables that we're using.

In Task 2, the decision tree achieved 68.53% mean cross-validation accuracy and 68.28% accuracy on the held-out test set. Categorical Naïve Bayes achieved 67.28% test accuracy.

For Q5, we ran k-means with 3, 4, and 5 clusters. With 5 clusters, the SSE was lowest at 323,705.25, suggesting households were, in general, closest to their cluster centers than with any other option we considered. SSE generally decreases as you increase the number of clusters, so we need to review the cluster profiles before describing the groups and assessing their usefulness.

4.2 Changes to Our Research Questions

We are keeping Questions 1 to 4 because they still contribute to the project's general aim: examining the relationships between household characteristics, vehicle ownership, and travel behaviour. At this stage, we examined Question 4 directly through classification. Questions 1 to 3 remain relevant for further analysis, and our work so far provides a basis for investigating them. Analyzing household vehicle ownership gave us experience preparing predictors, encoding categorical variables, and assessing classification models; these steps can therefore guide the preparation of a future model for transportation-mode prediction. However, transportation mode is a different target, so it will need its own model and evaluation.

Q2 is still fine. PCA showed that trip distance and trip duration loaded heavily on the same principal component, so that provides a basis for looking at the relationship between the two in more detail. To answer the original question, you'd still need to compare a model with distance only to a model that also includes transport mode, trip purpose, and day of week.

We kept Question 3 as it was. In Phase 1, we described vehicle age and annual mileage. We still need to examine how annual mileage varies with vehicle age, vehicle type, household income, and the number of workers in the household.

We've amended Question 5 to account for the clustering analysis in Phase 2. We originally intended to define household travel-day profiles using household size, number of vehicles, number of drivers, number of workers, and number of trips. That would have let us examine both household composition and the amount of travel recorded on the travel day.

In Phase 2, we used the same seven household predictors chosen for classification: household size, driver count, worker count, income category, homeownership, the number of young children, and urban classification. By using the same inputs, we could compare how classification and clustering handled the same household information.

We left out vehicle count because we used it in Task 2 to define the four ownership classes. We wanted to see whether households grouped by the other seven characteristics would fall into similar ownership classes. Similarly, we didn't include the recorded trip count, so the resulting clusters describe the household's demographic and socio-economic profile rather than its travel-day activity. That's why "household profiles" is a better description of what we have here than "travel day profiles".

4.3 Revised Research Question 5

Can we form meaningful household groups based on household size, number of drivers, number of workers, income category, homeownership, number of young children, and urban classification? To what extent do these groups correspond to the vehicle ownership classes?

The revised question consists of two sections. The first concerns whether clustering can identify groups of households with similar characteristics. To evaluate this, we must examine the cluster profiles and determine which attributes distinguish the groups. For instance, differences in household size, driver count, or worker count might help to explain the groupings, but any such descriptions must be based on the actual cluster results.

The second section examines whether these groups match the vehicle ownership categories. Our results indicated only limited agreement between cluster membership and the existing classes. This implies that similarity in household characteristics does not directly result in the same vehicle ownership category.

In our evaluation, the decision tree predicted vehicle ownership classes more successfully; clustering, by contrast, examined similarities between households without using the ownership labels during fitting. The revised Question 5 clarifies this difference and ensures our research question remains aligned with the variables, methods, and results in Phase 2.

We included the code, data, and results in 217761263-217285287-219714815.zip. The workbook 217761263-217285287-219714815.xlsx contains the Task 1 scores in PCA_3PCs and the Task 2 household predictors, class labels, and train/test split in Classification_Data.